Università degli Studi di Messina

DIPARTIMENTO DI SCIENZE MATEMATICHE E INFORMATICHE, SCIENZE FISICHE E SCIENZE DELLA TERRA

Bachelor's Degree in Computer Science L-31

Curriculum: Informatica

LAB-01: Introduzione al Machine Learning

Supervisore:

Prof. Giacomo Fiumara

Studenti:

Aurora Fabio – 558520

Mario Parisi – 555657

Sandi Russo – 553675

Anno Accademico

2025/2026

Indice

1 Attività A **1**

1 Attività A1 1

1.1 A1.1 1

1.2 A1.2 1

2 Attività A2 2

2.1 A2.1 2

2.2 A2.2 3

2 Attività B **4**

1 Attività B1 4

1.1 B1.1 4

2 Attività B2 5

2.1 B2.1 5

2.2 B2.2 5

2.3 B2.3 5

3 Attività B4 7

3.1 B4.1 7

3.2 B4.2 8

3.3 B4.3 8

3 Attività C **9**

1 C1 9

2 C2 10

2.1 C2.1 10

2.2 C2.2 10

2.3 C2.3 11

4 Attività D **12**

1 D1 12

1.1 D1.1 12

1.2 D1.2 12

1.3 D1.3 12

2 D2 13

2.1 D2.1 – Definizioni 13
2.2 D2.2 13
2.3 D2.3 13

5 Mini-Report di Gruppo **15**
1 Tabella classificazione degli approcci ML 15
2 Workflow applicato all'esempio GDP/life satisfaction 16
3 Fonti di bias e over/underfitting 16
4 Pitfalls – Problemi critici di un progetto ML 17

Attività A

1 Attività A1

1.1 A1.1

Un programma impara dall'esperienza E in relazione a un task T e a una misura di performance P, se le sue prestazioni in T, misurate da P, migliorano con l'esperienza E.

Applicazione	**Task** T	**Experience** E	**Performance** P
Filtro antispam	serve a classificare le email in arrivo assegnando a ciascuna un etichetta che identifichi che la mail sia spam o meno	training set, ovvero l'addestramento della macchina tramite un database di email già etichettate manualmente dagli utenti o tramite regole preimpostate.	la percentuale di email etichettate/classificate
Riconoscimento vocale	Trascrizione di un audio in testo	un dataset che contiene le registrazioni vocali di diverse persone con accenti e rumori di sottofondo e le rispettive trascrizioni di testo corrette	la percentuale di comandi vocali eseguiti con successo dall'app mobile
Sistema di raccomandazione	suggerire agli utenti dei prodotti che hanno una alta probabilità di essere acquistate in base alle preferenze degli utenti	l'elenco/storico degli acquisti degli utenti: acquisti precedenti, ricerche effettuate, prodotti visualizzati	l'aumento complessivo delle vendite e la percentuale di click da parte degli utenti sui prodotti suggeriti
Rilevazione frodi	determinare in tempo reale se una specifica transazione con carta di credito è legittima o fraudolenta	lo storico di milionidi di transazioni bancarie passate, che includono dati come: importo, orario, posizione geografica e, fondamentale, un'etichetta che certifica se la transazione si è rilevata una frode o meno	il numero di frodi effettivamente bloccate e l'ammontare di denaro risparmiato. Anche qui è cruciale minimizzare i falsi positivi (bloccare la carta a un cliente onesto che sta semplicemente facendo un acquisto insolito)

Tabella 1.1: Esempi di sistemi ML descritti secondo il framework T/E/P.

1.2 A1.2

- **Filtro antispam su e-mail**: **Classificazione binaria** – ogni e-mail in ingresso viene etichettata come spam/non-spam.

- **Riconoscimento vocale per comandi vocali in un'app mobile**: **Classificazione multiclasse** – il modello ascolta l'audio e lo classifica in una delle categorie di comandi già conosciute.
- **Sistema di raccomandazione prodotti per un e-commerce**: **Clustering**.
- **Rilevazione di frodi su carte di credito**: **Anomaly detection** – il sistema memorizza le caratteristiche delle operazioni/transazioni valide ed evidenzia come anomalia tutte quelle operazioni che si allontanano dallo schema conosciuto.

2 Attività A2

Filtro antispam con regole fatte a mano

Un sistema sviluppato tramite programmazione convenzionale prevede un processo iterativo e manuale:

- **Analisi del dominio**: il problema viene studiato per identificare le caratteristiche tipiche delle mail indesiderate; si individua la ricorrenza di termini specifici come "baby", "credit card", "banca".
- **Implementazione delle regole**: si sviluppa un algoritmo di rilevamento per ciascun pattern identificato; il programma è istruito per classificare un'email come spam qualora venga rilevato un determinato numero di tali pattern.
- **Valutazione e iterazione**: il programma viene testato e i passaggi precedenti vengono ripetuti finché le prestazioni non raggiungono uno standard accettabile per il rilascio.
- **Criticità**: a causa della complessità del problema, il sistema tende a trasformarsi in una lunga e complessa lista di regole, risultando difficilmente manutenibile.

Filtro antispam con approccio ML

- **Apprendimento dai dati**: il modello apprende automaticamente quali parole o frasi costituiscano buoni predittori di spam.
- **Rilevamento dei pattern**: l'algoritmo rileva pattern di parole che si presentano con una frequenza insolita negli esempi di spam rispetto alle email legittime.
- **Vantaggi strutturali**: il programma risultante si presenta molto più conciso, più semplice da mantenere e, con elevata probabilità, caratterizzato da una maggiore accuratezza.

2.1 A2.1

L'adozione di un sistema di Machine Learning in questo contesto è giustificata da diversi fattori tecnici e operativi:

- **Manutenibilità e sintesi del codice**: a differenza dell'approccio tradizionale, che richiede lunghe liste di regole, un modello ML consente di semplificare notevolmente il codice, facilitandone la manutenzione.

- **Maggiore accuratezza e prestazioni**: i sistemi di apprendimento automatico tendono a fornire prestazioni superiori e una maggiore accuratezza rispetto alle soluzioni basate su regole statiche.
- **Adattabilità ad ambienti fluttuanti (dinamicità)**: un sistema ML è in grado di adattarsi automaticamente ai cambiamenti. Qualora i mittenti di spam modifichino le proprie tattiche (ad esempio, sostituendo "4U" con "For U"), l'algoritmo rileva autonomamente la nuova frequenza anomala e aggiorna i propri criteri di classificazione senza richiedere alcun intervento manuale da parte dello sviluppatore.

2.2 A2.2

Un esempio emblematico in cui l'approccio tradizionale fallisce è il **riconoscimento vocale**.

- **Motivazione dell'inefficacia delle regole statiche**: si potrebbe tentare di scrivere un algoritmo basato su regole fisse, ad esempio misurando l'intensità dei suoni acuti per distinguere l'inizio della parola "two" rispetto alla parola "one". Tuttavia, tale tecnica non è scalabile qualora applicata a migliaia di parole pronunciate da milioni di individui differenti, in ambienti rumorosi e in decine di lingue diverse.
- **Soluzione**: in tali scenari di elevata complessità, la soluzione ottimale consiste nell'impiego di un algoritmo capace di apprendere autonomamente a partire da un vasto corpus di registrazioni di esempio.

Attività B

1 Attività B1

Esempio (Applicazione)	Supervisione	Output	Tipo di Task
Analisi immagini di prodotti sulla linea di produzione	Supervised	Classi	Image classification
Rilevamento tumori in brain scans	Supervised	Classi (per singolo pixel)	Semantic image segmentation
Classificazione automatica di articoli di news	Supervised	Classi	Text classification
Riconoscimento vocale per comandi vocali (app)	Supervised / Self-supervised	Altro (sequenze audio convertite)	Speech recognition
Rilevamento frodi su carte di credito	Unsupervised	Altro (normale vs anomalia)	Anomaly detection
Segmentazione clienti in base agli acquisti	Unsupervised	Classi (cluster/gruppi)	Clustering
Raccomandazione prodotti basata su acquisti passati	Supervised / Self-supervised	Valori (probabilità del prossimo acquisto)	Recommender system
Bot intelligente per un videogioco	Reinforcement Learning	Altro (azioni per massimizzare la ricompensa)	Reinforcement learning

Tabella 2.1: Classificazione degli esempi di Machine Learning per tipo di supervisione, output e task.

1.1 B1.1

- **Anomaly detection vs Novelty detection**: l'anomaly detection (es. frodi) viene addestrata mostrando al sistema istanze prevalentemente normali per fargli riconoscere le anomalie, mentre la novelty detection richiede un set di addestramento "pulito" e privo di qualsiasi istanza che l'algoritmo dovrebbe poi rilevare come novità.

- **Recommender systems**: sebbene possa sembrare clustering, un sistema di raccomandazione spesso utilizza reti neurali addestrate su sequenze di acquisti passati per prevedere in output l'acquisto successivo più probabile per l'utente.

2 Attività B2

2.1 B2.1

- **Supervised learning**: l'algoritmo viene addestrato fornendogli un set di dati che include già le soluzioni desiderate, denominate etichette (*labels*).
- **Unsupervised learning**: il sistema riceve dati di addestramento non etichettati e tenta di apprendere autonomamente la struttura e le connessioni nascoste senza la guida di un "insegnante".
- **Semi-supervised learning**: l'algoritmo gestisce dati parzialmente etichettati, combinando tipicamente tecniche non supervisionate (es. clustering) per raggruppare istanze simili, ed estendendo poi le poche etichette disponibili all'intero gruppo.
- **Self-supervised learning**: il sistema genera automaticamente un dataset etichettato a partire da dati non etichettati (es. mascherando parti di un'immagine) e usa un algoritmo supervisionato per prevedere la parte mancante.
- **Reinforcement learning**: un agente osserva un ambiente, intraprende azioni e riceve ricompense (o penalità), apprendendo autonomamente la strategia (*policy*) migliore per massimizzare il profitto nel tempo.

2.2 B2.2

La porzione **unsupervised** (clustering) si verifica quando il servizio analizza le foto e riconosce automaticamente che una stessa persona appare in più immagini, raggruppandole senza sapere chi sia. La porzione **supervised** (o semi-supervised) subentra nel momento in cui l'utente fornisce una singola etichetta (il nome) per persona, permettendo al sistema di propagare tale informazione e nominare automaticamente gli individui in tutte le foto del cluster.

2.3 B2.3

(i) **Generazione di input e label**: si seleziona un dataset di immagini non etichettate e si maschera casualmente una piccola porzione di ciascuna; l'immagine mascherata diventa l'input, mentre l'immagine originale e intatta viene utilizzata come etichetta (*target*).

(ii) **Perché non è puramente unsupervised**: sebbene operi su dataset inizialmente privi di etichette umane, questo approccio genera e utilizza etichette (le immagini originali) durante la fase di addestramento, risolvendo task tipici dell'apprendimento supervisionato come la classificazione o la regressione.

B3. Instance-based vs Model-based Learning

Gli algoritmi di apprendimento automatico supervisionato possono essere classificati in due macro-categorie in base alla strategia con cui generalizzano dai dati di addestramento: i modelli **instance-based** e i modelli **model-based**.

Un algoritmo **instance-based** non apprende una funzione esplicita dai dati: memorizza l'intero insieme di addestramento e, al momento della predizione, cerca i campioni più simili alla nuova osservazione per derivarne la risposta. L'algoritmo **k-NN** *k-Nearest Neighbors*) è l'esempio più rappresentativo di questa categoria: data una nuova istanza, individua i k punti più vicini nello spazio delle feature e ne calcola la media (per la regressione) o la moda (per la classificazione). Questo approccio è flessibile e non fa assunzioni sulla forma della relazione tra feature e target, ma è vulnerabile al rumore nei dati di addestramento e non è in grado di generalizzare al di fuori dell'intervallo osservato.

Un algoritmo **model-based**, al contrario, apprende una funzione parametrica che descrive la relazione tra feature e target, ottimizzando i parametri in modo da minimizzare un criterio di errore globale. La **regressione lineare** è il caso più semplice: apprende i coefficienti m e q della retta $\hat{y} = mx + q$ che meglio si adatta ai dati di addestramento secondo il criterio dei minimi quadrati. Una volta appresa la regola, il modello è in grado di generalizzare anche al di fuori dell'intervallo di addestramento.

Attività pratica: confronto su dataset sintetico

Per illustrare concretamente la differenza tra i due approcci è stato generato un dataset sintetico composto da 100 punti distribuiti uniformemente nell'intervallo $[0, 10]$, con valori target definiti dalla relazione lineare $y = mx + q$ (con m e q interi casuali nell'intervallo $[1, 4]$) e perturbati da rumore gaussiano con deviazione standard pari a 2. Il generatore di numeri casuali è stato inizializzato con seed fisso (`np.random.seed(42)`) per garantire la riproducibilità dell'esperimento.

I due modelli sono stati addestrati sullo stesso dataset rumoroso:

- **k-NN con** $k = 3$: per ogni punto di predizione calcola la media dei 3 campioni di addestramento più vicini.
- **Regressione lineare**: stima i coefficienti $\hat{m}$ e $\hat{q}$ che minimizzano l'errore quadratico medio sull'intero dataset.

I risultati, illustrati in Figura 2.1, evidenziano chiaramente le differenze comportamentali dei due approcci. La regressione lineare produce una retta regolare che approssima bene la tendenza globale dei dati, ignorando le fluttuazioni locali dovute al rumore. Il modello k-NN, invece, segue fedelmente le variazioni locali del dataset di addestramento, risultando più sensibile al rumore. Il divario tra i due approcci diventa particolarmente evidente agli estremi dell'intervallo di predizione $[-2, 12]$: la regressione lineare estrapola correttamente la tendenza lineare, mentre il k-NN si appiattisce nella zona di confine, privo di punti vicini su cui basare la predizione.

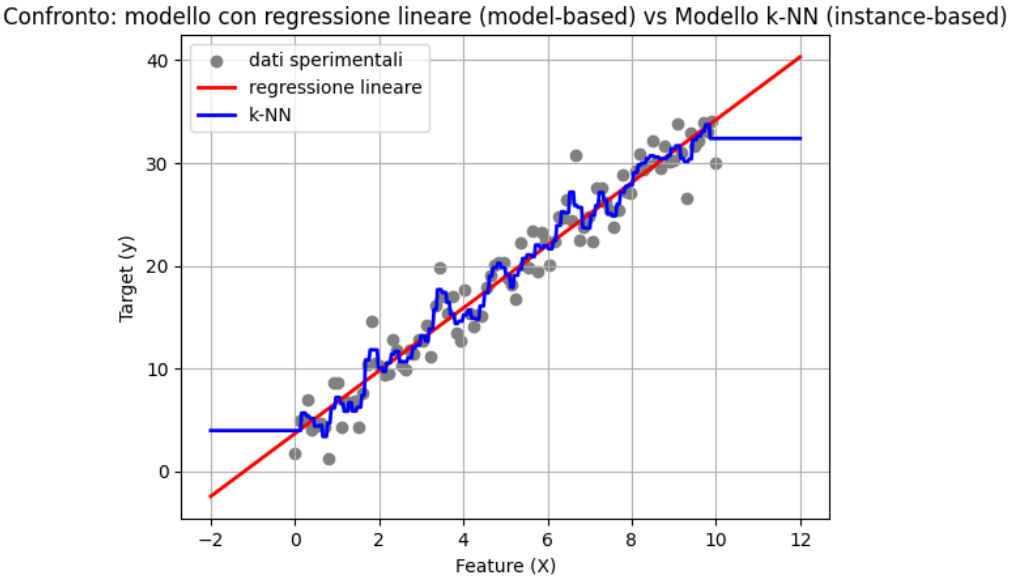

Figura 2.1: Confronto tra regressione lineare (model-based) e k-NN (instance-based) su dataset sintetico con rumore gaussiano

Questo esempio mostra come la scelta tra un approccio instance-based e uno model-based non sia neutrale: quando si ha ragione di credere che i dati seguano una struttura regolare (come una relazione lineare), un modello parametrico è preferibile perché generalizza meglio e risulta robusto al rumore. Il k-NN è invece più adatto quando la relazione tra feature e target è locale e non parametrizzabile con una forma funzionale semplice.

3 Attività B4

3.1 B4.1

In un progetto di apprendimento automatico, si raccolgono i dati in un set di addestramento e quest'ultimo viene alimentato a un algoritmo di apprendimento, che si basa su:

- **Modelli**: regola alcuni parametri per adattare il modello al set di addestramento; di conseguenza, sarà in grado di fare buone previsioni anche su nuovi casi. *Esempio*: soddisfazione della vita in base al PIL pro capite del paese – si nota che la soddisfazione di vita aumenta più o meno linearmente all'aumentare del PIL pro capite.

- **Istanze**: impara gli esempi a memoria e generalizza a nuove istanze usando una misura di similarità per confrontarle con le istanze apprese. *Esempio*: filtro antispam, segnala tutte le mail simili a quelle già segnalate dall'utente come spam.

3.2 B4.2

Il modello **kNN** si basa sulla memorizzazione e sulla somiglianza. Memorizza gli esempi e non cerca una regola generale (*instance-based*): durante la fase di addestramento (`fit`) memorizza il dataset.

La **regressione lineare** cerca di trovare una regola generale che si adatti a tutti i dati tramite un modello matematico globale (*model-based*). Durante la fase di addestramento (`fit`), l'algoritmo analizza tutti i punti e cerca di avvicinarsi il più possibile a tutti i dati, minimizzando l'errore.

3.3 B4.3

I sistemi di apprendimento automatico si possono classificare in base alla loro capacità di generalizzazione: *model-based* e *instance-based*.

- **Memoria**:
  - *Model-based*: generalizza a partire dai dati; viene memorizzato solo il modello utilizzato.
  - *Instance-based*: funziona memorizzando esempi quando arrivano nuovi dati; richiede uno spazio di memoria per ogni singola istanza.
- **Tempo di predizione**:
  - *Model-based*: rapido dopo l'addestramento del modello.
  - *Instance-based*: lento, poiché effettua ricerche sulle istanze memorizzate.
- **Interpretabilità**:
  - *Model-based*: fornisce un'interpretabilità globale, spiegando le regole generali apprese dall'algoritmo.
  - *Instance-based*: fornisce un'interpretabilità locale, giustificando la predizione in base alla somiglianza con esempi specifici precedentemente memorizzati.

Attività C

1 C1

a **Studio e preparazione dei dati**: questa fase iniziale comprende il reperimento (download), la preparazione come l'unione di tabelle e la separazione delle feature dalle etichette, e la visualizzazione dei dati tramite grafi a dispersione al fine di individuare eventuali tendenze o pattern nascosti.

b **Selezione del modello**: consiste nell'individuare la tipologia di modello più adatta a rappresentare i dati studiati e nello specificarne l'architettura.

c **Addestramento**: in questa fase, l'algoritmo elabora i dati di addestramento per individuare i parametri ottimali del modello che minimizzano una specifica funzione di costo.

d **Predizione o inferenza**: il modello, ormai addestrato, viene impiegato per generare previsioni su dati mai visti in precedenza, con l'assunto e la speranza che sia in grado di generalizzare correttamente i pattern appresi.

Corrispondenza con il codice GDP

- **Studio e preparazione dei dati**: il codice esegue il caricamento dei dati, la separazione degli input (x) e dei target (y), e la visualizzazione esplorativa.

```
# Download and prepare the data
data_root = "https://github.com/ageron/data/raw/main/"
lifesat = pd.read_csv(data_root + "lifesat/lifesat.csv")
X = lifesat[["GDP per capita (USD)"]].values
y = lifesat[["Life satisfaction"]].values

# Visualize the data
lifesat.plot(kind='scatter', grid=True,x="GDP per capita (USD)", y="Life
    satisfaction")
plt.axis([23_500, 62_500, 4, 9])
plt.show()
```

- **Selezione del modello**: il codice istanzia l'oggetto relativo all'algoritmo scelto.

```
# Select a linear model
model = LinearRegression()
```

- **Addestramento del modello**: tramite il metodo *fit*, si avvia l'algoritmo che calcola i parametri ottimali minimizzando la funzione di costo sui dati forniti.

```
model.fit(X, y)
```

- **Predizione (inferenza)**: il modello addestrato viene interrogato tramite il metodo *predict* per stimare il valore di output relativo a un nuovo input.

```
# Make a prediction
X_new = [[37_655.2]]
```

2 C2

2.1 C2.1

Il modello GDP

In questo esempio si descrive un modello lineare addestrato su un set di paesi che escludeva nazioni con un PIL pro capite inferiore a 23.500 dollari o superiore a 62.500 dollari. Aggiungendo i paesi mancanti, i dati mostrano che i paesi molto ricchi non sembrano più felici dei paesi moderatamente ricchi, e viceversa alcuni paesi poveri sembrano più felici di molti paesi ricchi.

Problematica: utilizzando un set di addestramento non rappresentativo, si addestra un modello che difficilmente farà previsioni accurate, specialmente per i casi più estremi come i paesi più ricchi o i paesi più poveri.

Il sondaggio del Literary Digest (Elezioni presidenziali USA del 1936)

Il *Literary Digest* condusse un enorme sondaggio inviando questionari per posta a circa 10 milioni di persone. Ricevette 2,4 milioni di risposte e previde con grande sicurezza che Landon avrebbe ottenuto il 57% dei voti. Tuttavia, vinse Roosevelt con il 62%.

Problematica: il difetto risiedeva nel metodo di campionamento – gli indirizzi a cui inviare i sondaggi vennero presi da elenchi telefonici e liste di membri di club, che tendevano a favorire le persone più ricche, di conseguenza più inclini a votare per Landon. Un ulteriore problema fu che rispose meno del 25% delle persone contattate. Questo creò una distorsione, nota come *bias di non risposta*, in quanto escluse potenzialmente chi non si interessava di politica o chi non apprezzava la rivista, rendendo i dati inaffidabili.

2.2 C2.2

In un articolo pubblicato nel 2001, i ricercatori Microsoft Michele Banko ed Eric Brill hanno dimostrato che algoritmi di machine learning diversi, inclusi quelli semplici, ottenevano prestazioni quasi identiche in un problema complesso di disambiguazione del linguaggio naturale una volta ricevuti abbastanza dati. Hanno quindi suggerito che potrebbe essere opportuno riconsiderare il compromesso tra la spesa di tempo e denaro nello sviluppo di algoritmi rispetto allo sviluppo dei dati. L'idea che i dati contino più degli algoritmi fu poi ulteriormente diffusa da Peter Norvig in un articolo del 2009 intitolato *"The Unreasonable Effectiveness of Data"*. Tuttavia, raccogliere dati extra per l'addestramento non è sempre facile o economico: spesso richiede tempo, risorse e lavoro umano, e pertanto non bisogna abbandonare gli algoritmi del tutto.

2.3 C2.3

Un modello è una rappresentazione semplificata che scarta i dettagli superflui dei dati per riuscire a generalizzare su casi nuovi. Di conseguenza, quando si sceglie un modello specifico, si stanno implicitamente facendo delle assunzioni sui dati stessi. Il teorema del *"No Free Lunch"*, formulato nel 1996 da David Wolpert, dimostra proprio che senza fare assunzioni sui dati, non esiste alcun motivo per preferire un modello rispetto a un altro. Poiché nessun modello ha la garanzia assoluta di funzionare a priori meglio degli altri in ogni situazione, per trovare quello perfetto bisognerebbe valutarli tutti. Dato che testare tutto è impossibile, nella pratica ci si affida a delle assunzioni ragionevoli sul problema per valutare solo un gruppo ristretto e adeguato di modelli.

Attività D

1 D1

1.1 D1.1

- **Overfitting (Sovradattamento)**: questo fenomeno si verifica quando un modello ottiene prestazioni eccellenti sui dati di addestramento (*training data*), ma si rivela incapace di generalizzare correttamente su nuove istanze mai viste in precedenza.
  Cause tipiche: l'overfitting si manifesta quando il modello scelto è eccessivamente complesso rispetto alla quantità e alla rumorosità dei dati di addestramento a disposizione. Se il set di dati è troppo piccolo o rumoroso, il modello tende a rilevare e memorizzare i pattern intrinseci del rumore (*sampling noise*) o variazioni dovute al puro caso, i quali non sono validi per la popolazione generale.

- **Underfitting (Sottoadattamento)**: rappresenta il fenomeno opposto all'overfitting e si verifica quando il modello risulta troppo semplice per riuscire ad apprendere la struttura sottostante dei dati. Di conseguenza, il modello produce predizioni inaccurate persino sugli stessi esempi utilizzati per l'addestramento.
  Cause tipiche: la causa principale risiede in una realtà dei dati intrinsecamente più complessa rispetto alla capacità rappresentativa del modello scelto (ad esempio, l'applicazione di un modello lineare a un fenomeno non lineare).

1.2 D1.2

Per diminuire il problema dell'overfitting e migliorare la capacità di generalizzazione del modello, ci sono tre strategie fondamentali:

- **Semplificazione del modello**: è possibile selezionare un modello con un numero inferiore di parametri (ad esempio, preferendo un modello lineare a uno polinomiale di grado elevato). In alternativa, si può ridurre il numero di attributi (*feature*) presenti nei dati di addestramento o applicare dei vincoli al modello stesso.

- **Raccolta di più dati**: incrementare la quantità dei dati di addestramento aiuta il modello a distinguere i pattern reali dal rumore casuale.

- **Riduzione del rumore nei dati di addestramento**: è opportuno pulire i dati, ad esempio correggendo eventuali errori di misurazione e rimuovendo i valori anomali (*outlier*).

1.3 D1.3

La **regolarizzazione** consiste nell'imporre dei vincoli a un modello al fine di renderlo più semplice e, conseguentemente, limitare il rischio di overfitting.

- **Esempio dei gradi di libertà**: in un modello di regressione lineare elementare caratterizzato da due parametri (θ_0 e θ_1), l'algoritmo di apprendimento dispone di "due gradi di libertà" per adattarsi ai dati di training: può modificare liberamente sia l'intercetta (θ_0) che la pendenza (θ_1) della retta.
- **Riduzione rigida**: qualora si forzasse $\theta_1 = 0$, l'algoritmo verrebbe limitato a un solo grado di libertà; in tal caso, potrebbe unicamente traslare una retta orizzontale verso l'alto o verso il basso, ottenendo un modello estremamente semplice ma con enormi difficoltà nell'adattarsi correttamente ai dati.
- **Il ruolo della regolarizzazione**: se si consente all'algoritmo di modificare θ_1, ma contemporaneamente lo si forza a mantenere tale valore molto piccolo (vincolo di regolarizzazione), l'algoritmo opererà con una flessibilità intermedia, compresa tra uno e due gradi di libertà. Questo processo produce un modello più semplice rispetto a uno con due gradi di libertà completi, ma più complesso di uno con un solo grado: si raggiunge così il giusto equilibrio tra adattamento ai dati di training e buona generalizzazione su nuovi esempi.

2 D2

2.1 D2.1 – Definizioni

- **Training set**: l'insieme di esempi che il sistema utilizza per apprendere e addestrare il modello.
- **Validation set** (detto anche *development set* o *dev set*): una porzione del training set che viene tenuta da parte per valutare diversi modelli candidati e selezionare il migliore; in pratica, si addestrano più modelli con vari iperparametri sulla parte restante del training set e si sceglie quello che ottiene le prestazioni migliori sul validation set.
- **Test set**: un insieme di dati separato utilizzato per testare il modello finale, permettendo di ottenere una stima dell'errore di generalizzazione e di capire come il sistema si comporterà su istanze che non ha mai visto prima.

2.2 D2.2

Se si misura ripetutamente l'errore di generalizzazione sul test set per scegliere il modello e i suoi iperparametri, si finisce per adattare questi elementi in modo specifico per performare al meglio proprio su quel particolare insieme di dati. Il risultato è che il modello diventerà troppo specializzato per quel test set e, di conseguenza, sarà improbabile che riesca a ottenere prestazioni altrettanto buone su dati nuovi.

2.3 D2.3

Il **train-dev set** è un ulteriore sottoinsieme di dati tenuto da parte dall'insieme di addestramento originale ed è particolarmente utile quando i dati usati per l'addestramento sono abbondanti ma non perfettamente rappresentativi dei dati reali che verranno usati in produzione. Dopo aver addestrato il modello sul training set ridotto, lo si valuta proprio sul train-dev set:

- Se le prestazioni sono scarse in questa fase, significa che il modello ha semplicemente fatto overfitting sul training set.
- Se invece il modello si comporta bene sul train-dev set, lo si valuta sul dev set. Se in questo caso le prestazioni calano drasticamente, significa che il problema non è l'overfitting, ma il **data mismatch**, ovvero la sostanziale differenza tra l'ambiente di training/addestramento e quello reale.

Mini-Report di Gruppo

1 Tabella classificazione degli approcci ML

Dimensione	**Possibili categorie**
Supervisione	**Supervised**: classificazione (filtro spam) **Unsupervised**: clustering (divisione dei gruppi in base all'età) **Semi-supervised**: mix tra supervised e unsupervised (Google Foto) **Self-supervised**: classificazione multiclasse (classificazione degli animali: data un'immagine in input si ottiene la specie dell'animale presente nella foto) **Reinforcement learning**: tramite un agente stabilisce la migliore strategia (*policy*) per ricevere più ricompense possibili (robot che aspira e lava il pavimento)
Flusso di apprendimento	**Batch**: il sistema deve essere addestrato su tutti i dati (classificazione di animali aggiornato periodicamente da zero) **Online**: apprendimento incrementale fornendo dati in sequenza (rilevamento di nuovi schemi nel mercato azionario) **Out-of-core**: addestramento a blocchi su dataset grandi lavorando offline
Meccanismo di generalizzazione	**Instance-based**: il sistema apprende tramite gli esempi passati, generalizzando i nuovi casi e comparandoli ai precedenti (regressione kNN) **Model-based**: generalizzare gli esempi per costruire un modello dal quale fare previsioni (regressione lineare)

Tabella 5.1: Dimensioni di classificazione dei sistemi di Machine Learning.

2 Workflow applicato all'esempio GDP/life satisfaction

Il workflow applicato all'esempio di GDP/life satisfaction segue le 4 fasi di un progetto di Machine Learning:

1. **Studiare i dati**: abbiamo scaricato i dati del Better Life Index dell'OCSE e il PIL dalla Banca Mondiale. Tramite *matplotlib* con il tipo *scatter* abbiamo unito e visualizzato questi dati.

```
matplotlib  # funzione .plot(kind='scatter')
```

2. **Selezionare il modello**: visto che i dati sembravano crescere in modo lineare, abbiamo selezionato il modello di regressione lineare con due parametri θ_0 e θ_1.

```
model = LinearRegression()
```

3. **Addestrare il modello**: abbiamo definito una funzione di costo per misurare le distanze tra le predizioni e i dati reali e minimizzarla, ottenendo $\theta_0 = 3{,}75$ e $\theta_1 = 6{,}78 \times 10^{-5}$.

```
model.fit(X, y)
```

4. **Fare previsioni (Inferenza)**: abbiamo usato il modello finale per stimare la soddisfazione di vita di un nuovo paese, Cipro, inserendo il suo PIL pro capite.

```
model.predict(X_new)
```

3 Fonti di bias e over/underfitting

In questo esempio, il modello lineare era addestrato su un set di paesi che inizialmente escludeva nazioni con un PIL pro capite molto basso (inferiore a 23.500) o molto alto. Aggiungendo i dati mancanti, possiamo notare che i paesi molto ricchi non sembrano più felici dei paesi moderatamente ricchi, e viceversa alcuni paesi poveri sembrano più felici di molti paesi ricchi: ciò rappresenta una **fonte di bias**.

A causa dell'**overfitting**, il modello non capisce lo schema generale, ma memorizza i dettagli e i "rumori" dei dati di addestramento. Nel caso del GDP/life satisfaction, l'aggiunta di feature irrilevanti causa overfitting.

A causa dell'**underfitting**, si registrano errori molto alti sia in fase di training che di testing. Nel caso del GDP/life satisfaction, ciò si manifesta come un'inaccuratezza elevata sui dati di addestramento stessi.

4 Pitfalls – Problemi critici di un progetto ML

I problemi più critici di un progetto di Machine Learning sono:

- **Insufficiente quantità di dati di addestramento**: per problemi più semplici, gli algoritmi hanno bisogno di migliaia di esempi; per problemi più complessi come il riconoscimento vocale o di immagini, potrebbero servire anche milioni di esempi.
  Soluzione: riusare parti di un modello già esistente (*transfer learning*).

- **Dati di addestramento non rappresentativi**: inizialmente il set di dati del GDP/life satisfaction non conteneva i dati sui paesi con PIL inferiore a 23.500 o superiore a 62.500 dollari. Questi dati non rappresentativi comportano predizioni poco accurate.
  Soluzione: migliorare il metodo di campionamento per evitare distorsioni (*nonresponse bias*).

- **Dati di scarsa qualità**: se i dati di addestramento sono pieni di errori o rumore, per il sistema sarà complicato rilevare i sottostanti schemi.
  Soluzione: dedicare tempo alla pulizia dei dati, scartando gli outlier e correggendo o ignorando gli errori.

- **Overfitting dei dati di addestramento**: il modello performa in modo ottimo sui dati di addestramento ma non generalizza bene.
  Soluzione: semplificare il modello, raccogliere più dati di addestramento e ridurne il rumore. Il processo di restrizione e semplificazione del modello è chiamato **regolarizzazione**.

- **Underfitting dei dati di addestramento**: il modello è troppo semplice per imparare la struttura sottostante dei dati.
  Soluzione: selezionare un modello più potente con più parametri, fornire feature migliori all'algoritmo (*feature engineering*) e ridurre i vincoli imposti sul modello.